

Two Main Binary Search Patterns

There are basically two families:

1. Search for exact target (classic BS)

Used when you want to find an element's index.

```
int l = 0, h = n-1;
while(l <= h){
    int m = l + (h-l)/2;
    if(arr[m] == target) return m; // found exact
    else if(arr[m] < target) l = m+1; // move right
    else h = m-1; // move left
}
return -1; // not found
```

 Rule here:

- `while(l <= h)` because you want to check the last element.
 - Updates: `l = m+1` or `h = m-1` (you discard mid after checking).
 - Return depends: usually `return m` inside when found.
-

2. Search for boundary (lower/upper bound style)

Used when you want the **smallest/largest value** that satisfies a condition (`canEat`, `canFinish`, etc).

- **If minimizing** $\rightarrow h = m$.
- **If maximizing** $\rightarrow l = m$.

```
int l = 1, h = sum;
while(l < h){
    int m = l + (h-l)/2;
    if(canEat(piles, hrs, m)) {
        h = m; // keep mid, look left (minimization)
    } else {
        l = m+1; // discard mid, need bigger speed
    }
}
return l; // l == h at end
```

 Rule here:

- `while(l < h)` because you stop when range shrinks to one.

- Updates:
 - If condition true (too big, can optimize more) $\rightarrow h = m$.
 - If condition false (too small, need bigger) $\rightarrow l = m+1$.
 - Return is l (or h , both same at end).
-

Trick to Decide Update Rule

Ask yourself:

2. **Am I searching for an element (equality)?** $\rightarrow \text{while}(l \leq h), l = m+1 / h = m-1$.
 3. **Am I searching for a boundary (min/max satisfying condition)?**
 - If minimizing $\rightarrow h = m$.
 - If maximizing $\rightarrow l = m$.
-

Example Variants

Variant A – Minimize speed

```
while(l < h){
    int m = (l+h)/2;
    if(canEat(..., m)) h = m;    // shrink right
    else l = m+1;
}
return l; // min speed
```

Variant B – Maximize speed

```
while(l < h){
    int m = (l+h+1)/2; // bias up
    if(canEat(..., m)) l = m; // keep mid, move right
    else h = m-1;
}
return l; // max speed
```

 Notice:

- For **minimization** $\rightarrow h = m$
 - For **maximization** $\rightarrow l = m$
 - And adjust m (+1 sometimes) to avoid infinite loop.
-

Golden Rule

- $l = m+1$ or $h = m-1 \rightarrow$ use when mid is already **invalid** (discard mid).
- $l = m$ or $h = m \rightarrow$ use when mid is still **possible** (keep mid).

- `while(l < h) →` for boundary problems.
- `while(l <= h) →` for exact search.

1 Classic Binary Search (Exact Match / Search Insert / Occurrences)

1. Binary Search to Find X in Sorted Array

Problem Statement:

Given a sorted array `arr[]` of size `n` and an integer `x`, determine if `x` exists in the array. Return its index if found, otherwise -1.

Test Case:

Input: `arr = [1,3,5,7,9]`, `X = 5`
Output: 2

Input: `arr = [2,4,6,8,10]`, `X = 1`
Output: -1

Pattern / Variation: Classic binary search, exact match.

How to Identify: Array is **sorted**, want exact element index.

Clean C++ Code:

```
int binarySearch(vector<int>& arr, int X) {
    int lo = 0, hi = arr.size() - 1;
    while(lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if(arr[mid] == X) return mid;
        else if(arr[mid] < X) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

2. Implement Lower Bound

Problem Statement:

Given a sorted array `arr[]` and a target `x`, find the **index of the first element $\geq X$** . If no such element exists, return `n` (size of array).

Lower bound = *min index where $arr[i] \geq X \rightarrow$ minimization problem*

Test Case:

Input: `arr = [1,3,5,7,9]`, `X = 4`
Output: 2 // `arr[2] = 5`

Input: `arr = [2,4,6,8]`, `X = 10`
Output: 4 // beyond array

Pattern / Variation: Lower Bound

How to Identify: Sorted array, first element $\geq$ target.

Clean C++ Code:

```
int lowerBound(vector<int>& arr, int X) {  
    int l = 0, h = arr.size(); // notice h = n (not n-1)  
    while (l < h) {  
        int mid = l + (h - l) / 2;  
        if (arr[mid] >= X) {  
            h = mid; // mid can still be the answer  
        } else {  
            l = mid + 1; // discard left part including mid  
        }  
    }  
    return l; // or h (both are equal here)  
}
```

or

```
int lowerBound(vector<int>& arr, int X) {  
    int lo = 0, hi = arr.size() - 1, ans = arr.size();  
    while(lo <= hi) {  
        int mid = lo + (hi - lo) / 2;  
        if(arr[mid] >= X) { ans = mid; hi = mid - 1; }  
        else lo = mid + 1;  
    }  
    return ans;  
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Here we do:

```
int l = 0, h = n;    // not n-1
```

Reason:

- In this variant, ***h is treated as an exclusive bound*** (search space is $[l, h)$).
- Meaning the loop runs while $l < h$.
- The valid indexes are $0 \dots n-1$, but we set $h = n$ so that the last index $n-1$ is included in the range.

If we set $h = n - 1$ here, the last index could be skipped (especially in cases where answer = n, like when x is greater than all elements).

3. Implement Upper Bound

Problem Statement:

Given a sorted array `arr[]` and a target `x`, find the **index of the first element $> X$** . If no such element exists, return `n`.

Test Case:

Input: `arr = [1,3,5,7]`, `X = 5`

Output: 3 // `arr[3] = 7`

Input: `arr = [2,4,6,8]`, `X = 10`

Output: 4

Pattern / Variation: Upper Bound

How to Identify: Sorted array, first element $>$ target.

Clean C++ Code:

```
int upperBound(vector<int>& arr, int X) {
    int l = 0, h = arr.size();    // h = n (exclusive)
    while (l < h) {
        int mid = l + (h - l) / 2;
        if (arr[mid] > X) {
            h = mid;    // minimize index (first greater than X)
        } else {
            l = mid + 1;
        }
    }
}
```

```

    }
    return l; // or h (they are equal here)
}

```

Or

```

int upperBound(vector<int>& arr, int X) {
    int lo = 0, hi = arr.size() - 1, ans = arr.size();
    while(lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if(arr[mid] > X) { ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans;
}

```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Why $h = n$ again?

- Because answer might be n (if $X \geq$ all elements).
 - Example: $arr = [2, 4, 6], X = 10 \rightarrow$ answer should be 3.
 - If you used $h = n-1$, loop wouldn't allow that case $\rightarrow$ wrong result.
-

Memory Trick:

- **lowerBound ($\geq X$)** $\rightarrow$ condition: `if (arr[mid] >= X) h = mid; else l = mid+1;`
 - **upperBound ($> X$)** $\rightarrow$ condition: `if (arr[mid] > X) h = mid; else l = mid+1;`
 - Start with $h = n$, end with `return l`.
-

So in short:

- For both **lowerBound** and **upperBound** in `while(l < h)` style $\rightarrow$ set $h = n$.
 - Just change the comparison ($\geq$ vs $>$)
-

4. Search Insert Position

Problem Statement:

Given a sorted array `arr[]` and target `x`, return **the index where X should be inserted** to maintain sorted order.

Test Case:

Input: `arr = [1,3,5,6]`, `x = 5`
Output: 2

Input: `arr = [1,3,5,6]`, `x = 2`
Output: 1

Pattern / Variation: Same as Lower Bound

How to Identify: Sorted array, want position to insert element.

Clean C++ Code:

```
int searchInsert(vector<int>& arr, int X) {  
    return lowerBound(arr, X); // reuse lowerBound function  
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

5. Floor / Ceil in Sorted Array

Problem Statement:

- **Floor:** Largest element $\leq X$ ----- `floor(4.2) = 4`
- **Ceil:** Smallest element $\geq X$ ----- `ceil(4.2) = 5`

Return index for floor or ceil, or -1 if not exists.

Test Case:

`arr = [1,3,5,7]`, `x = 4`
Floor index = 1 (`arr[1]=3`), Ceil index = 2 (`arr[2]=5`)

`arr = [1,3,5,7]`, `x = 0`
Floor index = -1, Ceil index = 0

Pattern / Variation: Floor / Ceil

How to Identify: Sorted array, closest $\leq$ or $\geq$ target.

Clean C++ Code:

```
int floorIndex(vector<int>& arr, int X) {
```

```

    int lo = 0, hi = arr.size() - 1, ans = -1;
    while(lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if(arr[mid] <= X) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return ans;
}

int ceilIndex(vector<int>& arr, int X) {
    int lo = 0, hi = arr.size() - 1, ans = arr.size();
    while(lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if(arr[mid] >= X) { ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans == arr.size() ? -1 : ans;
}

```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

6. First or Last Occurrence

Problem Statement:

Given sorted array `arr[]` with duplicates, find **first** or **last** occurrence of `x`.

Test Case:

`arr = [1,2,2,2,3], X = 2`
 First occurrence = 1, Last occurrence = 3

Pattern / Variation: First/Last Occurrence

How to Identify: Array sorted with duplicates, want boundary index.

Clean C++ Code:

```

int firstOccurrence(vector<int>& arr, int X) {
    int lo=0, hi=arr.size()-1, ans=-1;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(arr[mid] == X){ ans = mid; hi = mid - 1; }
        else if(arr[mid] < X) lo = mid + 1;
        else hi = mid - 1;
    }
    return ans;
}

int lastOccurrence(vector<int>& arr, int X) {
    int lo=0, hi=arr.size()-1, ans=-1;

```

```

while(lo <= hi) {
    int mid = lo + (hi - lo)/2;
    if(arr[mid] == X){ ans = mid; lo = mid + 1; }
    else if(arr[mid] < X) lo = mid + 1;
    else hi = mid - 1;
}
return ans;
}

```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

OR

◆ First Occurrence

First occurrence of x = **lower bound of x**
(lower bound gives *first index where $arr[i] \geq X$*).

```

int firstOccurrence(vector<int>& arr, int X) {
    int idx = lower_bound(arr.begin(), arr.end(), X) - arr.begin();
    if (idx < arr.size() && arr[idx] == X) return idx;
    return -1;
}

```

◆ Last Occurrence

Last occurrence of x = **upper bound of $x - 1$**
(upper bound gives *first index where $arr[i] > X$*).

```

int lastOccurrence(vector<int>& arr, int X) {
    int idx = upper_bound(arr.begin(), arr.end(), X) - arr.begin();
    idx--; // move one step left
    if (idx >= 0 && arr[idx] == X) return idx;
    return -1;
}

```

✓ Example

Array = [2, 4, 4, 4, 6, 8], $X = 4$

- `lower_bound(arr, arr+n, 4)` → index 1 (first 4)
- `upper_bound(arr, arr+n, 4)` → index 4 (first > 4) → last occurrence = $4 - 1 = 3$.

7. Count Occurrences of a Number

Problem Statement:

Given sorted array with duplicates, count how many times x appears.

Test Case:

```
arr = [1,2,2,2,3,4], X = 2  
Output = 3
```

Pattern / Variation: Count duplicates

How to Identify: Array sorted, want number of duplicates.

Clean C++ Code:

```
int countOccurrences(vector<int>& arr, int X) {  
    int first = firstOccurrence(arr, X);  
    if(first == -1) return 0; // not found  
    int last = lastOccurrence(arr, X);  
    return last - first + 1;  
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

2 Binary Search in Rotated Sorted Array

Pattern: Array is sorted but rotated; you may need to find target, min element, or rotation count.

Trick: Identify which half is sorted and decide which half to move.

1. Search in Rotated Sorted Array I

Problem Statement:

Given a sorted array `arr[]` rotated at an unknown pivot (no duplicates), find the index of target x . Return -1 if not found.

Test Case:

```
Input: arr = [4,5,6,7,0,1,2], X = 0  
Output: 4
```

Input: arr = [4,5,6,7,0,1,2], X = 3
Output: -1

Pattern / Variation: Search in rotated array, no duplicates.

How to Identify: Array sorted but rotated, target index required, no duplicates.

Clean C++ Code:

```
int searchRotated(vector<int>& arr, int X) {
    int lo = 0, hi = arr.size() - 1;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(arr[mid] == X) return mid;

        if(arr[lo] <= arr[mid]) { // left half sorted
            if(X >= arr[lo] && X < arr[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // right half sorted
            if(X > arr[mid] && X <= arr[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Short approach:

if arr[lo] is less then equal to arr[mid] that means left half is sorted :

so check if tgt X lies in left half by $X \geq \text{arr[lo]}$ && $X < \text{arr[mid]}$ if yes **hi = mid - 1**(move to left)

if not look at right half **lo = mid + 1**

else: right half is sorted

check if it lies in right half $X > \text{arr[mid]}$ && $X \leq \text{arr[hi]}$

if yes go to right : **lo = mid + 1**

if no go to left : **hi = mid - 1**

2. Search in Rotated Sorted Array II

Problem Statement:

Same as above, but **duplicates allowed**. Return index of target or -1.

Test Case:

Input: arr = [2,5,6,0,0,1,2], X = 0

Output: 3

Input: arr = [2,5,6,0,0,1,2], X = 3

Output: -1

Pattern / Variation: Search in rotated array, duplicates present.

How to Identify: Array rotated, may contain duplicates, target search.

Clean C++ Code:

```
int searchRotatedWithDuplicates(vector<int>& arr, int X) {
    int lo = 0, hi = arr.size() - 1;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(arr[mid] == X) return mid;

        // skip duplicates
        while(lo < mid && arr[lo] == arr[mid]) lo++;

        if(arr[lo] <= arr[mid]) { // left sorted
            if(X >= arr[lo] && X < arr[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // right sorted
            if(X > arr[mid] && X <= arr[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Time Complexity: O(n) worst case (due to duplicates), O(log n) average

Space Complexity: O(1)

3. Find Minimum in Rotated Sorted Array

Problem Statement:

Given a rotated sorted array arr[] (no duplicates), find the **minimum element**.

Test Case:

Input: arr = [4,5,6,7,0,1,2]

Output: 0

Input: arr = [3,4,5,1,2]
Output: 1

Pattern / Variation: Find min element in rotated array.

How to Identify: Array rotated, want minimum value.

Minimization problem : hi = mid

Clean C++ Code:

```
int findMinRotated(vector<int>& arr) {
    int lo = 0, hi = arr.size() - 1;
    while(lo < hi) {
        int mid = lo + (hi - lo)/2;
        if(arr[mid] > arr[hi]) lo = mid + 1; // min in right half
        else hi = mid; // min in left half
    }
    return arr[lo];
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

4. Find How Many Times Array Rotated

Problem Statement:

Given a sorted array **arr[]** (in strictly increasing order) that has been right-rotated **k** times. A right rotation means the last element is moved to the first position, and the remaining elements are shifted one position to the right. Find the value of **k** the number of times the array was right-rotated from its originally sorted form.

Given a rotated sorted array, find **number of rotations** (index of minimum element).

Test Case:

Input: arr = [15,18,2,3,6,12]
Output: 2 // arr[2] = 2 is min

Input: arr = [7,9,11,12,5]
Output: 4

Pattern / Variation: Rotation count = index of min element.

How to Identify: Array rotated, find index of min.

Clean C++ Code:

```
int countRotations(vector<int>& arr) {
```

```

int lo = 0, hi = arr.size() - 1;
while(lo < hi) {
    int mid = lo + (hi - lo)/2;
    if(arr[mid] > arr[hi]) lo = mid + 1;
    else hi = mid;
}
return lo;
}

```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

✓ Summary / Tricks

Pattern	Trick
Search in Rotated Array	Compare $arr[mid]$ with $arr[lo]/arr[hi]$ to detect sorted half
With Duplicates	Skip duplicates if $arr[lo]==arr[mid]$
Find Min	Compare $arr[mid]$ with $arr[hi]$, move lo/hi accordingly
Count Rotations	Same as Find Min, return index

3 Single Element / Peak in 1D Array

Pattern: Array is mostly duplicate/ordered; find unique element or peak.

Trick: Compare neighbors or use index parity to reduce search space.

1. Single Element in a Sorted Array

Problem Statement:

Given a sorted array `arr[]` where **every element appears twice except one element** that appears only once, find that single element.

Test Case:

Input: `arr = [1,1,2,2,3,4,4]`
Output: 3

Input: `arr = [0,0,1,1,2]`
Output: 2

Pattern / Variation: Single element, others twice.

How to Identify: Array sorted, all elements repeated except one, find that single element.

Clean C++ Code (Index Parity Method):

```
int singleNonDuplicate(vector<int>& arr) {
    int lo = 0, hi = arr.size() - 1;
    while(lo < hi) {
        int mid = lo + (hi - lo)/2;
        if(mid % 2 == 1) mid--; // make mid even
        if(arr[mid] == arr[mid + 1]) lo = mid + 2; // single element is after
pair, right side
        else hi = mid; // single element is before or at mid
    }
    return arr[lo];
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

2. Find Peak Element

Problem Statement:

Given an array `arr[]`, find a **peak element**. A peak element is **strictly greater than its neighbors**. Return the index of any peak.

Test Case:

Input: `arr = [1,2,3,1]`
Output: 2 // `arr[2] = 3`

Input: `arr = [1,3,2,1]`
Output: 1 // `arr[1] = 3`

Pattern / Variation: Peak in 1D array.

How to Identify: Element larger than neighbors, array unsorted or bitonic.

Clean C++ Code:

```
int findPeakElement(vector<int>& arr) {
    int lo = 0, hi = arr.size() - 1;
    while(lo < hi) {
        int mid = lo + (hi - lo)/2;
        if(arr[mid] < arr[mid + 1]) lo = mid + 1; // peak on right
        else hi = mid; // peak on left or mid
    }
    return lo;
}
```

Time Complexity: $O(\log n)$
Space Complexity: $O(1)$

3. Find Square Root of a Number (Integer $\sqrt{N}$)

Problem Statement:

Given an integer N , find the **integer part of its square root** (largest integer $\leq \sqrt{N}$).

Test Case:

Input: $N = 10$
Output: 3

Input: $N = 16$
Output: 4

Pattern / Variation: Binary search in value space.

How to Identify: Find $\sqrt{N}$ or similar monotonic function.

Clean C++ Code:

```
int integerSqrt(int N) {
    int lo = 0, hi = N, ans = 0;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(1LL * mid * mid <= N) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return ans;
}
```

Time Complexity: $O(\log N)$

Space Complexity: $O(1)$

4. Find Nth Root of a Number

Problem Statement:

Given integers x and N , find the **integer part of Nth root of X** (largest integer R such that $R^N \leq x$).

Test Case:

Input: $x = 16, N = 2$
Output: 4

Input: $X = 27, N = 3$
Output: 3

Input: $X = 15, N = 2$
Output: 3

Pattern / Variation: Binary search in value space.

How to Identify: Monotonic function $X^{(1/N)}$, want integer root.

Clean C++ Code:

```
int nthRoot(int X, int N) {
    int lo = 1, hi = X, ans = 1;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        long long power = 1;
        for(int i = 0; i < N; i++) {
            power *= mid;
            if(power > X) break;
        }
        if(power <= X) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return ans;
}
```

Time Complexity: $O(\log X * N)$

Space Complexity: $O(1)$

☑ Summary / Tricks for Pattern 3

Problem	Trick
Single element	Use index parity; check pairs
Peak element	Compare mid with mid+1; go to increasing side
Square root	Binary search value space
Nth root	Binary search value space, multiply mid^N

4 Binary Search on Answer / Monotonic Function (Optimization Problems)

Pattern: Find minimum/maximum value satisfying a condition.

Trick: Define a **monotonic predicate** and binary search on answer/value space.

1. Koko Eating Bananas

Koko loves to eat bananas. There are n piles of bananas, the i^{th} pile has $\text{piles}[i]$ bananas. The guards have gone and will come back in h hours.

Koko can decide her bananas-per-hour eating speed of k . Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return the minimum integer k such that she can eat all the bananas within h hours.

Example 1:

Input: $\text{piles} = [3,6,7,11]$, $h = 8$

Output: 4

Example 2:

Input: $\text{piles} = [30,11,23,4,20]$, $h = 5$

Output: 30

Example 3:

Input: $\text{piles} = [30,11,23,4,20]$, $h = 6$

Output: 23

[100,100,1,1] , $h = 4$

Output : 100

Constraints:

- $1 \leq \text{piles.length} \leq 10^4$
- $\text{piles.length} \leq h \leq 10^9$
- $1 \leq \text{piles}[i] \leq 10^9$

Pattern / Variation: Binary search on answer (minimum eating speed).

How to Identify: Problem asks for min/max satisfying condition.

Intuition:

- Speed $\uparrow \rightarrow$ Time $\downarrow$ (monotonic).
- Predicate: check if $\text{sum}(\text{ceil}(p/k)) \leq h$.

Clean C++ Code:

```
bool canEat(vector<int>& piles, int K, int H) {
    int hours = 0;
    for(int pile : piles) hours += (pile + K - 1) / K; // ceil division
    return hours <= H;
}

int minEatingSpeed(vector<int>& piles, int H) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end()), ans = hi;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(canEat(piles, mid, H)) { ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans;
}
```

Time Complexity: $O(n * \log(\text{maxPile}))$

Space Complexity: $O(1)$

2. Minimum Days to Make M Bouquets

You are given an integer array `bloomDay`, an integer `m` and an integer `k`.

You want to make `m` bouquets. To make a bouquet, you need to use `k` adjacent flowers from the garden.

The garden consists of `n` flowers, the i^{th} flower will bloom in the `bloomDay[i]` and then can be used in exactly one bouquet.

Return the minimum number of days you need to wait to be able to make `m` bouquets from the garden. If it is impossible to make `m` bouquets return -1.

Example 1:

Input: bloomDay = [1,10,3,10,2], m = 3, k = 1

Output: 3

Explanation: Let us see what happened in the first three days. x means flower bloomed and _ means flower did not bloom in the garden.

We need 3 bouquets each should contain 1 flower.

After day 1: [x, _ , _ , _ , _] // we can only make one bouquet.

After day 2: [x, _ , _ , _ , x] // we can only make two bouquets.

After day 3: [x, _ , x, _ , x] // we can make 3 bouquets. The answer is 3.

Example 2:

Input: bloomDay = [1,10,3,10,2], m = 3, k = 2

Output: -1

Explanation: We need 3 bouquets each has 2 flowers, that means we need 6 flowers. We only have 5 flowers so it is impossible to get the needed bouquets and we return -1.

Example 3:

Input: bloomDay = [7,7,7,7,12,7,7], m = 2, k = 3

Output: 12

Explanation: We need 2 bouquets each should have 3 flowers.

Here is the garden after the 7 and 12 days:

After day 7: [x, x, x, x, _ , x, x]

We can make one bouquet of the first three flowers that bloomed. We cannot make another bouquet from the last three flowers that bloomed because they are not adjacent.

After day 12: [x, x, x, x, x, x, x]

It is obvious that we can make two bouquets in different ways.

Constraints:

- `bloomDay.length == n`
- `1 <= n <= 105`
- `1 <= bloomDay[i] <= 109`
- `1 <= m <= 106`
- `1 <= k <= n`

Intuition:

- If we can make bouquets on day `d`, then also on any day `> d`.  monotonic.
- Binary search on days.

Clean C++ Code:

```
bool canMake(vector<int>& bloom, int days, int m, int k) {
    int count = 0, consecutive = 0;
    for(int b : bloom) {
        if(b <= days) consecutive++;
        else consecutive = 0;
        if(consecutive == k) { count++; consecutive = 0; }
    }
    return count >= m;
}

int minDays(vector<int>& bloom, int m, int k) {
    int lo = 1, hi = *max_element(bloom.begin(), bloom.end()), ans = -1;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(canMake(bloom, mid, m, k)) { ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans;
}
```

Time Complexity: $O(n * \log(\text{maxDay}))$

Space Complexity: $O(1)$

3. Find the Smallest Divisor

Problem Statement:

Given array `nums[]` and `threshold`, find the **smallest divisor** such that sum of `ceil(nums[i]/divisor) ≤ threshold`.

Test Case:

Input: nums = [1,2,5,9], threshold = 6
Output: 5

Clean C++ Code:

```
bool isValid(vector<int>& nums, int div, int threshold) {
    int sum = 0;
    for(int num : nums) sum += (num + div - 1) / div;
    return sum <= threshold;
}

int smallestDivisor(vector<int>& nums, int threshold) {
    int lo = 1, hi = *max_element(nums.begin(), nums.end()), ans = hi;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(isValid(nums, mid, threshold)) { ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans;
}
```

Time Complexity: $O(n * \log(\text{maxNum}))$

Space Complexity: $O(1)$

4. Capacity to Ship Packages within D Days

Problem Statement:

Given `weights[]` and `D` days, find **minimum ship capacity** to ship all packages in `D` days. Packages must be shipped in order.

Test Case:

Input: weights = [1,2,3,4,5,6,7,8,9,10], D = 5
Output: 15

Clean C++ Code:

```
bool canShip(vector<int>& weights, int cap, int D) {
    int days = 1, curr = 0;
    for(int w : weights) {
        if(curr + w > cap) { days++; curr = 0; }
        curr += w;
    }
    return days <= D;
}

int shipWithinDays(vector<int>& weights, int D) {
    int lo = *max_element(weights.begin(), weights.end()),
```

```

        hi = accumulate(weights.begin(), weights.end(), 0), ans = hi;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        if(canShip(weights, mid, D)) { ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans;
}

```

Time Complexity: $O(n * \log(\text{sumWeights}))$

Space Complexity: $O(1)$

5. Kth Missing Positive Number

Given an array `arr` of positive integers sorted in a strictly increasing order, and an integer `k`.

Return the k^{th} positive integer that is missing from this array.

Example 1:

Input: `arr = [2,3,4,7,11]`, `k = 5`

Output: 9

Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing positive integer is 9.

Example 2:

Input: `arr = [1,2,3,4]`, `k = 2`

Output: 6

Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive integer is 6.

Constraints:

- $1 \leq \text{arr.length} \leq 1000$
- $1 \leq \text{arr}[i] \leq 1000$
- $1 \leq k \leq 1000$

- $arr[i] < arr[j]$ for $1 \leq i < j \leq arr.length$

Clean C++ Code:

```
int missingElement(vector<int>& arr, int K) {
    int lo = 0, hi = arr.size() - 1;
    while(lo <= hi) {
        int mid = lo + (hi - lo)/2;
        int missing = arr[mid] - (mid + 1);
        if(missing < K) lo = mid + 1;
        else hi = mid - 1;
    }
    return lo + K;
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

✓ Trick / Pattern Summary

- **Koko / Minimum Days / Divisor / Ship Packages** → Binary search **answer space**, predicate monotonic.
- **Kth Missing** → Binary search on **count missing numbers** $\leq mid$.

1 Aggressive Cows (Maximize Minimum Distance)

Problem Statement:

You are given N stalls positions in a line and C cows. Place cows in stalls such that the **minimum distance** between any two cows is maximized. Return that maximum minimum distance.

Input:

Stalls = [1, 2, 8, 4, 9], $C = 3$
Output = 3

Explanation → Place cows at 1, 4, 8 (min distance = 3).

Pattern:

Binary Search on Answer (maximize minimum distance).

Trick: Predicate → Can we place C cows if min distance $\geq mid$?

Intuition:

If cows can be placed at distance `mid`, try larger. Else reduce distance.

C++ Code:

```
#include <bits/stdc++.h>
using namespace std;

bool canPlace(vector<int>& stalls, int cows, int dist) {
    int count = 1, last = stalls[0];
    for (int i = 1; i < stalls.size(); i++) {
        if (stalls[i] - last >= dist) {
            count++;
            last = stalls[i];
        }
        if (count >= cows) return true;
    }
    return false;
}

int aggressiveCows(vector<int>& stalls, int cows) {
    sort(stalls.begin(), stalls.end());
    int low = 1, high = stalls.back() - stalls[0], ans = 0;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (canPlace(stalls, cows, mid)) ans = mid, low = mid + 1;
        else high = mid - 1;
    }
    return ans;
}

int main() {
    vector<int> stalls = {1, 2, 8, 4, 9};
    int cows = 3;
    cout << aggressiveCows(stalls, cows);
}
```

TC: $O(N \log(\text{maxDist}))$

SC: $O(1)$

2 Book Allocation Problem

Problem Statement:

You have N books and M students. Each student must get **contiguous books**. Minimize the maximum pages allocated to a student.

Input:

Books = [12, 34, 67, 90], M = 2
Output = 113

Explanation → Allocate [12, 34, 67] and [90]. Max = 113.

Pattern:

Binary Search on Answer (minimize maximum pages).

Predicate → Can books be divided into $\leq M$ students if max allowed = mid?

Intuition:

Check with greedy partitioning.

C++ Code:

```
#include <bits/stdc++.h>
using namespace std;

bool canAllocate(vector<int>& books, int students, int limit) {
    int count = 1, sum = 0;
    for (int pages : books) {
        if (pages > limit) return false;
        if (sum + pages > limit) {
            count++;
            sum = pages;
        } else sum += pages;
    }
    return count <= students;
}

int allocateBooks(vector<int>& books, int students) {
    int low = *max_element(books.begin(), books.end());
    int high = accumulate(books.begin(), books.end(), 0), ans = high;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (canAllocate(books, students, mid)) ans = mid, high = mid - 1;
        else low = mid + 1;
    }
    return ans;
}

int main() {
    vector<int> books = {12, 34, 67, 90};
    int students = 2;
    cout << allocateBooks(books, students);
}
```

TC: $O(N \log(\text{sum}))$

SC: $O(1)$

3 Split Array - Largest Sum

Same as Book Allocation (just different statement).

Problem: Split array into k subarrays $\rightarrow$ minimize largest sum.

Input: $[7, 2, 5, 10, 8]$, $k=2 \rightarrow$ Output = 18.

(Split as $[7, 2, 5]$ and $[10, 8]$)

Code = Same as Book Allocation.

4 Painter's Partition

Problem: N boards of length `arr[i]` and k painters. Each painter paints **contiguous boards**. Each unit takes 1 time. Minimize total time.

Input: `boards=[10,20,30,40]`, $K=2 \rightarrow$ Output = 60.

(One painter: $[10,20,30]$, other: $[40]$)

Pattern: Exactly like Book Allocation.

Code: $\rightarrow$ Same as Book Allocation (just rename).

5 Minimize Max Distance to Gas Station

Problem:

We have `stations` on a line. We can add k more stations. Minimize the **maximum distance** between consecutive stations.

Input:

`Stations = [1, 2, 8, 12]`, $K = 2$
Output = 2.0

Pattern:

Binary Search on **real values**.

Predicate $\rightarrow$ Can we add $\leq K$ stations if max distance = mid?

Intuition:

If max allowed = mid, calculate how many stations needed in each gap.

C++ Code:

```
#include <bits/stdc++.h>
using namespace std;

bool canAdd(vector<int>& st, int k, double dist) {
    int need = 0;
    for (int i = 1; i < st.size(); i++) {
        double gap = st[i] - st[i-1];
        need += (int)(gap / dist); // stations needed
    }
    return need <= k;
}

double minimizeMaxDist(vector<int>& st, int k) {
    double low = 0, high = st.back() - st[0], ans = high;
    for (int it = 0; it < 60; it++) { // precision loop
        double mid = (low + high) / 2;
        if (canAdd(st, k, mid)) ans = mid, high = mid;
        else low = mid;
    }
    return ans;
}

int main() {
    vector<int> st = {1, 2, 8, 12};
    int k = 2;
    cout << fixed << setprecision(2) << minimizeMaxDist(st, k);
}
```

TC: $O(N \log(1/\epsilon))$ (binary search over precision)

SC: $O(1)$

6 Median of Two Sorted Arrays

Problem:

Find median of two sorted arrays nums1, nums2.

Input: nums1=[1,4,6], nums2=[2,3,5,7] → Output = 4.0

nums1=[1,3], nums2=[2,4,5,6] → Output = 4.0

Pattern:

Binary Search on Partition Index.

Intuition:

Partition both arrays such that left half = right half size, and all left $\leq$ right.

C++ Code:

```
#include <bits/stdc++.h>
using namespace std;

double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
    if (A.size() > B.size()) return findMedianSortedArrays(B, A);
    int n = A.size(), m = B.size();
    int low = 0, high = n;

    while (low <= high) {
        int cutA = (low + high) / 2;
        int cutB = (n + m + 1) / 2 - cutA;

        int leftA = (cutA == 0) ? INT_MIN : A[cutA-1];
        int rightA = (cutA == n) ? INT_MAX : A[cutA];
        int leftB = (cutB == 0) ? INT_MIN : B[cutB-1];
        int rightB = (cutB == m) ? INT_MAX : B[cutB];

        if (leftA <= rightB && leftB <= rightA) {
            if ((n+m)%2==0)
                return (max(leftA, leftB) + min(rightA, rightB)) / 2.0;
            else
                return max(leftA, leftB);
        }
        else if (leftA > rightB) high = cutA - 1;
        else low = cutA + 1;
    }
    return 0.0;
}

int main() {
    vector<int> A = {1,3}, B = {2};
    cout << findMedianSortedArrays(A, B);
}
```

TC: $O(\log(\min(n, m)))$

SC: $O(1)$

[1 Search in a 2D Matrix \(Fully sorted\)](#)

Problem

You are given an $m \times n$ matrix where each row is sorted and the first element of each row is greater than the last element of the previous row.
Find if a target exists.

Example

```
Matrix:
[ [1, 3, 5, 7],
  [10, 11, 16, 20],
  [23, 30, 34, 60] ]
target = 3 → true
target = 13 → false
```

Pattern

- **Variation:** Treat 2D as 1D
- **Identify:** Fully sorted across rows and columns.

Intuition

- Flatten to virtual 1D ($\text{row} = \text{mid}/n, \text{col} = \text{mid}\%n$).
- Do normal binary search.

Code

```
bool searchMatrix(vector<vector<int>>& mat, int target) {
    int m = mat.size(), n = mat[0].size();
    int low = 0, high = m * n - 1;

    while (low <= high) {
        int mid = low + (high - low)/2;
        int val = mat[mid/n][mid%n];
        if (val == target) return true;
        else if (val < target) low = mid + 1;
        else high = mid - 1;
    }
    return false;
}
```

Complexity

- **TC:** $O(\log(m*n))$
- **SC:** $O(1)$

2 Search in Row & Column Wise Sorted Matrix

Problem

Matrix rows & columns sorted (like Leetcode 240).
Check if target exists.

Example

```
Matrix:
[ [1, 4, 7, 11],
  [2, 5, 8, 12],
  [3, 6, 9, 16],
  [10,13,14,17] ]
target = 5 → true
target = 20 → false
```

Pattern

- **Variation:** Row+Col sorted
- **Identify:** If top-right strategy works.

Intuition

- Start **top-right**.
- If $\text{target} < \text{current}$ → move left.
- If $\text{target} > \text{current}$ → move down.

Code

```
bool searchMatrix(vector<vector<int>>& mat, int target) {
    int m = mat.size(), n = mat[0].size();
    int r = 0, c = n - 1;

    while (r < m && c >= 0) {
        if (mat[r][c] == target) return true;
        else if (mat[r][c] > target) c--;
        else r++;
    }
    return false;
}
```

Complexity

- **TC:** $O(m+n)$
- **SC:** $O(1)$

3 Find Peak Element in 2D Matrix

Problem

A **peak element** is one that is strictly greater than its 4 neighbors (up, down, left, right).
Find **any one peak position**.

Example

Matrix:
[[1, 4],
 [3, 2]]
Possible peaks: (0,1) with value 4

Pattern

- **Variation:** 2D peak search
- **Identify:** Move towards greater neighbor.

Intuition

- Binary search on columns.
- Pick column mid, find max element in that column.
- Compare with left/right column, move to side with greater neighbor.

Code

```
vector<int> findPeakGrid(vector<vector<int>>& mat) {
    int m = mat.size(), n = mat[0].size();
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high)/2;
        // find max row in this col
        int row = 0;
        for (int i = 0; i < m; i++)
            if (mat[i][mid] > mat[row][mid]) row = i;

        bool left = (mid - 1 >= 0 && mat[row][mid-1] > mat[row][mid]);
        bool right = (mid + 1 < n && mat[row][mid+1] > mat[row][mid]);

        if (!left && !right) return {row, mid};
        else if (left) high = mid - 1;
        else low = mid + 1;
    }
    return {-1, -1};
}
```

Complexity

- **TC:** $O(m \log n)$
 - **SC:** $O(1)$
-

4 Matrix Median

Problem

Matrix is row-wise sorted. Find median of all elements.

Example

```
Matrix:
[ [1, 3, 5],
  [2, 6, 9],
  [3, 6, 9] ]
Median = 5
```

Pattern

- **Variation:** Binary search on **value space**.
- **Identify:** Row-wise sorted & asking **median/min/max** → predicate works.

Intuition

- Range of values = [min, max].
- Binary search a number `mid`.
- Count how many elements $\leq \text{mid}$ using `upper_bound`.
- If count $\leq (m*n)/2$ → increase low else decrease high.

Code

```
int matrixMedian(vector<vector<int>>& mat) {
    int m = mat.size(), n = mat[0].size();
    int low = INT_MAX, high = INT_MIN;
    for (int i = 0; i < m; i++) {
        low = min(low, mat[i][0]);
        high = max(high, mat[i][n-1]);
    }

    int desired = (m*n + 1)/2;
    while (low < high) {
        int mid = (low + high)/2;
        int cnt = 0;
        for (int i = 0; i < m; i++)
            cnt += upper_bound(mat[i].begin(), mat[i].end(), mid) -
mat[i].begin();
        if (cnt < desired) low = mid + 1;
        else high = mid;
    }
    return low;
}
```

Complexity

- **TC:** $O(m \log(\max - \min) * \log n)$
 - **SC:** $O(1)$
-

✓ Now you have **all 4 binary search on 2D patterns**:

1. Fully sorted (flatten $\rightarrow$ 1D BS)
2. Row+col sorted (staircase search)
3. 2D Peak (BS on cols)
4. Matrix Median (BS on value space)